TÀI LIỆU KỸ THUẬT SỐ 02 / BỘ 04 TÀI LIỆU TRIỂN KHAI

KẾ HOẠCH HUẤN LUYỆN MÔ HÌNH (MODEL TRAINING PLAN)

Lộ trình huấn luyện từ bản Demo nhanh (SFT + RFT) đến Học tăng cường (RL) hiện đại cho các kiến trúc nhỏ

> **Chiến lược huấn luyện 2 chặng (Đi từ dễ đến khó):** Thay vì lao ngay vào huấn luyện Học tăng cường (RL) phức tạp vốn tốn nhiều thời gian căn chỉnh, kế hoạch chia làm **Chặng 1 (Dựng bản Demo nhanh)** chỉ dùng tinh chỉnh có giám sát (SFT) và lọc mẫu tự sửa lỗi (RFT) để có hệ thống chạy thử ngay; sau đó mới bước sang **Chặng 2 (Tối ưu hóa SOTA RL & LLM-JEPA)** để đẩy độ chính xác vượt qua mô hình 7B.

1. Huấn Luyện Agent 1 (Grounding — Bộ Tìm Bảng & Cột)

1.1. Chặng Demo: Huấn luyện bộ nhúng Qwen3-Embedding-0.6B (LitE-SQL)

Sử dụng gói dữ liệu `DATA-G1-EMB` (90,102 điểm dữ liệu cột trên BIRD), chúng ta tinh chỉnh mô hình `Qwen3-Embedding-0.6B` theo phương pháp của LitE-SQL **Piao et al. (2026)**:

- **Hàm mục tiêu huấn luyện:** Sử dụng hàm mất mát tương phản có giám sát lọc mẫu âm khó (HN-SupCon) với biên phân cách `margin = 0.1`. Hàm này kéo vector của câu hỏi lại gần vector của các cột đúng (`y = 1`) và đẩy thật xa vector của các cột trùng tên gây nhiễu trong cùng cơ sở dữ liệu (`y = 0`).
- **Đóng băng và tạo chỉ mục Offline:** Ngay sau khi train xong (đạt tỷ lệ tìm đúng cột `TPR = 95.23%` với top-25 cột), chúng ta đóng băng trọng số của mô hình 0.6B, chạy mã hóa một lượt toàn bộ các cột của các cơ sở dữ liệu và lưu thành file vector trên ổ cứng. Khi chạy thực tế, hệ thống chỉ tốn vài mili-giây để đối chiếu vector câu hỏi rồi đưa vào thuật toán **Cây Steiner tĩnh (Graph-Link)** **Zhong et al. (2026)** mà không cần train thêm mạng nào khác ở Chặng Demo.

1.2. Chặng Nâng Cao: Huấn luyện thêm mạng đồ thị Relation-Aware GNN

Ở Chặng 2 (dành cho Combo C và Combo D), lấy vector từ `Qwen3-Embedding-0.6B` làm đặc trưng khởi tạo cho các nút trên đồ thị `DATA-G1-GRAPH`, sau đó huấn luyện một mạng nơ-ron đồ thị nhẹ (Relation-Aware Heterogeneous GNN) để mỗi cột tự động học thêm thông tin từ các bảng hàng xóm nối qua khóa ngoại trước khi chạy thuật toán Cây Steiner.

2. Huấn Luyện Agent 2 — Bước 1: Tinh Chỉnh SFT & Căn Chỉnh LLM-JEPA

Ở Giai đoạn 2 (SQL Generation), chúng ta có 3 ứng viên sáng giá cho bản Demo là `SLM-SQL-1.5B`, `CodeS-3B` và `FINER-SQL-3B` (cùng mô hình `Qwen2.5-Coder-7B` cho Combo D). Quá trình dạy bước đầu tuân thủ hai kỹ thuật quan trọng:

2.1. Kỹ thuật "Chỉ tính điểm phạt trên phần trả lời" (Completion-Only Loss — Dùng ngay ở Demo)

Khi tinh chỉnh mô hình trên gói dữ liệu `DATA-G2-SFT` (gồm tập `SynSQL-Think-916K` và ngân hàng lời giải `Reasoning Bank`), một sai lầm phổ biến là tính hàm mất mát (loss) trên toàn bộ văn bản đầu vào. Theo phương pháp của FINER-SQL **Hoang et al. (2026)**, chúng ta đặt mặt nạ che (mask) toàn bộ phần đề bài (câu hỏi, câu lệnh tạo bảng DDL, gợi ý), **chỉ tính hàm mất mát trên phần mô hình tự viết ra** (tức chuỗi suy luận `<think>` và câu lệnh SQL). Cách làm này rất dễ cài đặt (chỉ cần gán nhãn `-100` cho các token đề bài trong PyTorch/HuggingFace) nhưng ngăn chặn triệt để việc mô hình nhỏ học vẹt tên bảng của từng cơ sở dữ liệu cụ thể.

2.2. Kỹ thuật căn chỉnh não bộ LLM-JEPA (Bổ sung ở Chặng 2 — Tăng +7.8% độ chính xác)

Khi nâng cấp lên Chặng 2 theo công trình của **Huang et al. (2026)**, ngoài việc tập viết từng từ SQL, chúng ta ép mô hình dùng vector ẩn của token dự đoán `[PRED]` nằm cuối khối Câu hỏi (Text) để dự đoán trúng vector ẩn tổng thể của khối Câu lệnh SQL (Code) thông qua khoảng cách cosine $d(\cdot,\cdot)$:

$$L_{LLM\text{-}JEPA} = \sum L_{LLM}(Text_{1:\ell-1}, Text_{\ell}) + \lambda \cdot d(Pred(Enc(Text)), Enc(Code))$$

- **Bộ thông số chuẩn đã kiểm chứng:** Đặt hệ số cân bằng $\lambda = 1$, số lượng token dự đoán $k = 1$, tốc độ học $lr = 2 \times 10^{-5}$, dùng mặt nạ chú ý nhân quả 2 khối (đặt giá trị $-\infty$ ở vùng giao nhau để khối Text và khối Code không nhìn trộm nhau).
- **Mẹo tiết kiệm một nửa thời gian train:** Bật chế độ **Random JEPA-Loss Dropout (`LD = 0.5`)** — tức cứ 2 lô dữ liệu (mini-batch) thì ngẫu nhiên chỉ tính hàm JEPA ở 1 lô, kết hợp huấn luyện bằng **LoRA ở rank 512** (chỉ cập nhật 22.59% số tham số). Thực nghiệm chứng minh cách này vẫn đạt độ chính xác ngang bằng với việc huấn luyện toàn bộ mô hình (tăng từ 51.95% lên 71.10% trên bài toán dịch sang mã có cấu trúc).

3. Huấn Luyện Agent 2 — Bước 2: Học Tăng Cường (RL) Giải Cứu Mô Hình Nhỏ

3.1. Tại sao cách train RL của MARS-SQL gốc bị "đứng hình" trên mô hình nhỏ?

Hệ thống MARS-SQL gốc dùng thuật toán **GRPO** (Group Relative Policy Optimization): với mỗi câu hỏi, mô hình tự viết thử một nhóm gồm G câu lệnh SQL, nhận điểm thưởng $\{r_1, r_2, ..., r_G\}$ rồi tính độ vượt trội tương đối trong nhóm theo công thức $A_j = (r_j - mean(r)) / std(r)$. Cách này tiết kiệm một nửa bộ nhớ GPU vì không cần mạng chấm điểm Critic.

Tuy nhiên, MARS-SQL gốc chỉ chấm điểm theo kết quả cuối cùng (chạy đúng được thưởng, chạy sai được 0 điểm). Khi áp dụng lên mô hình nhỏ (0.5B–3B) trước một câu hỏi khó của tập BIRD, **cả 8 lần viết thử của mô hình nhỏ đều bị lỗi và nhận 0 điểm (** $r_1 = r_2 = \ldots = r_G = 0$ **)** — cho dù có câu đã viết đúng được 90% và chỉ thiếu một dấu ngoặc. Khi mọi phương án đều bằng 0 điểm, độ vượt trội $A_j = 0$, đạo hàm biến mất hoàn toàn (hiện tượng *sụp đổ lợi thế / advantage collapse*) và mô hình ngừng học.

3.2. Năm phương pháp huấn luyện sửa lỗi & RL hiện đại tùy theo từng ứng viên

1. **Phương án dễ nhất cho Bản Demo — Rejection Fine-Tuning (RFT) của LitE-SQL Piao et al. (2026):** Không cần chạy GRPO phức tạp. Chỉ cần cho mô hình sau bước SFT chạy thử trên tập huấn luyện, nếu câu lệnh bị lỗi cơ sở dữ liệu thì cho mô hình đọc thông báo lỗi để tự viết lại. Những lượt tự sửa lỗi thành công (ra kết quả đúng) được lọc giữ lại để tinh chỉnh thêm một vòng có gắn bộ hãm phân kỳ $\beta \cdot D_{KL}(\pi_\theta \| \pi_0)$ nhằm tránh quên kiến thức cũ.

2. **Phương án SOTA cho mô hình 3B — GRPO 4 Phần Thưởng Dày của FINER-SQL Hoang et al. (2026):** Thay vì chấm đúng/sai nhị phân, điểm thưởng của mỗi lần thử là tổng của 4 đầu điểm chi tiết giúp các lần thử luôn có điểm chênh lệch nhau ($A_j \neq 0$):
$R(s|q) = R_{format}(s) + R_{exec}(s|q) + R_{mem}(s) + R_{atomic}(s, s^*)$
   - **Điểm trình bày ($R_{format} \in \{0, 1\}$):** Thưởng 1.0 điểm nếu đóng mở đúng thẻ `<think>` và thẻ SQL.
   - **Điểm chạy thử ($R_{exec} \in \{0, 1, 2\}$):** Được 0 điểm nếu báo lỗi cú pháp/runtime; được **1.0 điểm khuyến khích** nếu câu lệnh chạy trơn tru trên cơ sở dữ liệu dù kết quả chưa trúng; và được **2.0 điểm tối đa** nếu kết quả khớp hoàn toàn đáp án.
   - **Điểm suy nghĩ hợp lý ($R_{mem} \in [0, 1]$):** Khi câu SQL bị sai, hệ thống so sánh độ giống nhau về mặt ý nghĩa (cosine similarity của vector nhúng) giữa đoạn suy nghĩ `<think>` mà mô hình vừa viết với lời giải chuẩn trong `Reasoning Bank` để thưởng điểm tư duy đúng hướng.
   - **Điểm mảnh ghép toán tử ($R_{atomic} \in [0, 1]$):** Tách câu SQL dự đoán và câu SQL đáp án thành từng mảnh nhỏ (`SELECT`, `FROM`, `JOIN`, `WHERE`) rồi thưởng điểm theo tỷ lệ phần trăm số mảnh ghép viết đúng. Nhờ 4 điểm này, `FINER-SQL-3B` đạt **67.73% EX** trên BIRD-dev (vượt cả mô hình 7B cơ sở của MARS-SQL ở mức 66.37%).

3. **Phương án cho mô hình siêu nhỏ 0.5B–1.5B — Đồng huấn luyện Bộ Sinh & Bộ Gộp CSC-SQL Sheng & Xu (2025b):** Dùng cho Combo A. Huấn luyện GRPO cho 2 mô hình nhỏ phối hợp nhau: Mô hình 1 (`SLM-SQL-1.5B`) học viết 16 phương án SQL; Mô hình 2 (`Merge Revision 0.5B`, học trên gói `DATA-G2-MERGE`) được thưởng điểm khi đọc các phương án có kết quả đá nhau và gộp lại thành một câu lệnh đúng duy nhất (giúp bản 1.5B tăng vọt từ 28.40% lên **67.08% EX**).

4. **Phương án trị câu hỏi siêu khó — Chấm điểm từng bảng tạm CTE Zhang et al. (2026) & Gợi ý giảm dần COLM (2026):** Với những câu hỏi nối nhiều bảng dễ làm sai phép cộng gộp (`GROUP BY`), phương pháp **Reward-SQL** dạy mô hình chia câu lệnh dài thành từng bảng tạm `WITH ... AS` (Chain-of-CTEs) và dùng mô hình chấm điểm quá trình (PRM) thưởng ngay trên kết quả chạy của từng bảng tạm (giảm 82.4% lỗi `GROUP BY`). Hoặc dùng **Goal-Conditioned Hint Curriculum**: khi mô hình bí hoàn toàn ở chế độ tự làm (g_0), hệ thống mớm gợi ý từ nhiều đến ít ($g_3 \to g_2 \to g_1 \to g_0$) để mô hình luôn có mẫu giải được mà học dần.

5. **Phương án cho mô hình lớn 7B (Combo D) — Phần thưởng tối giản Arctic-Text2SQL-R1 Yao et al. (2025):** Riêng với mô hình 7B đã đủ thông minh, sau khi học SFT + LLM-JEPA, chỉ cần chạy GRPO với điểm thưởng thực thi và cú pháp tối giản để tránh bị nhiễu bởi các điểm thành phần phụ (giúp bản 7B đạt 68.90% EX trên BIRD-dev).

4. Huấn Luyện Agent 3 (Validation — Bộ Phán Quyết 0-Token)

Ở Giai đoạn 3, bộ kiểm tra 10 quy tắc cây cú pháp **PV-SQL** hoạt động bằng luật Python cố định nên **không cần huấn luyện**. Công việc huấn luyện chỉ tập trung vào các mô hình quyết định nhanh System-1 trên gói dữ liệu `DATA-G3-VAL` (128,576 quỹ đạo):

- **Tinh chỉnh miền SQL (Domain SFT) bắt buộc cho NanoJev-0.6B TianyuCodings (2026):** Bản `NanoJev-0.6B` gốc trên mạng được huấn luyện trên dữ liệu chơi game (ViZDoom, Maze, Snake). Nếu đem dùng thẳng cho bài toán chọn câu lệnh SQL (nơi 8 phương án thay đổi liên tục theo từng câu hỏi), độ chính xác bị sập từ 52.0% xuống chỉ còn **17.0%** (Macro-F1 = 0.249). Do đó, bắt buộc phải chạy tinh chỉnh có giám sát (Domain SFT) trên tập 16 quỹ đạo SQL/câu hỏi với 2 tốc độ học tách biệt: **khung xương ngôn ngữ đặt** $lr = 1\times10^{-5}$ và **đầu quyết định (Choice/Boolean Head) đặt** $lr = 1\times10^{-4}$. Chỉ sau bước SFT nhẹ này, độ chính xác chọn câu lệnh đúng của NanoJev tăng vọt lên **93.8%**.
- **Huấn luyện hiệu chuẩn xác suất (RLCD) cho Laya-421M ConvAI (2026):** Trên nền kiến trúc `ModernBERT-large` (nặng chỉ ~85 MB), chèn một token `[MASK]` đại diện cho mỗi phương án SQL để chấm điểm song song. Thay vì chỉ dùng hàm Cross-Entropy thông thường (dễ làm mô hình lúc nào cũng tự tin báo 99% dù chọn sai), chúng ta dùng thuật toán **RLCD (Reinforcement Learning for Calibrated Decisions)** phạt nặng những lần mô hình báo điểm tự tin cao nhưng câu SQL thực thi bị sai. Nhờ vậy, điểm tự tin của Laya-421M phản ánh đúng xác suất thực tế với sai số hiệu chuẩn cực thấp `ECE = 0.081`, cho phép đặt **Cổng dừng sớm ở ngưỡng tự tin** `0.85`.
- **Huấn luyện từ chối chủ động (Abstention RL) cho Kev-0.8B:** Sử dụng các mẫu câu hỏi mà cả 16 phương án sinh thử đều sai, thưởng điểm dương khi `Kev-0.8B` chọn hành động "Từ chối / Yêu cầu viết lại" và phạt nặng nếu cố tình chọn bừa một phương án sai, giúp giảm tỷ lệ đoán mò từ 26% xuống chỉ còn 5%.

5. Bảng Tổng Hợp Cấu Hình Huấn Luyện Cho Từng Mô-Đun

Mô-Đun Ứng Viên	Chặng 1: Bản Demo Nhanh	Chặng 2: Tối Ưu Hóa SOTA	Thông Số Kỹ Thuật Trọng Tâm
Agent 1: LitE-SQL (0.6B) + Graph-Link	Train HN-SupCon + Cây Steiner tĩnh trên đồ thị PK-FK	Train thêm mạng đồ thị Relation-Aware GNN	Biên `margin = 0.1`, lấy `top-k = 25` cột; lưu vector cột xuống ổ cứng (offline indexing).
Agent 2: SLM-SQL-1.5B / CodeS-3B	SFT trên `SynSQL-Think-916K` + Gắn khóa XGrammar PDA	LLM-JEPA + Đồng huấn luyện GRPO bộ gộp CSC-SQL (0.5B)	Completion-only loss; lấy mẫu 16 ứng viên + 8 mẫu gộp sửa lỗi khi bất đồng kết quả.
Agent 2: FINER-SQL-3B	SFT trên `Reasoning Bank` + Tự sửa lỗi RFT + XGrammar	LLM-JEPA + GRPO 4 phần thưởng dày (hoặc CoCTE / Hint)	LoRA rank 512, $lr = 2\times10^{-5}$, $\lambda = 1$, $k = 1$, $LD = 0.5$; 4 điểm: Format, Exec, Mem, Atomic.

Mô-Đun Ứng Viên	Chặng 1: Bản Demo Nhanh	Chặng 2: Tối Ưu Hóa SOTA	Thông Số Kỹ Thuật Trọng Tâm
Agent 3: NanoJev-0.6B / Laya-421M	PV-SQL 10 quy tắc AST + Domain SFT trên 16 quỹ đạo	Huấn luyện RLCD (Laya-421M) & Abstention RL (Kev-0.8B)	NanoJev: `lr` khung xương 1×10^{-5}, `lr` đầu quyết định 1×10^{-4}; Cổng dừng sớm ở ngưỡng `0.85`.

Tài Liệu Tham Khảo

1. Piao, S., Lee, J., & Park, S. (2026). *LitE-SQL: A Lightweight and Efficient Text-to-SQL Framework with Vector-based Schema Linking and Execution-Guided Self-Correction*. Findings of EACL 2026 (arXiv:2510.09014).
2. Zhong, J., et al. (2026). *Graph-Link: Bridging the Semantic-Structural Gap in Text-to-SQL via Constrained Subgraph Induction*. ICML 2026.
3. Hoang, T. D., et al. (2026). *FINER-SQL: Boosting Small Language Models for Text-to-SQL with Fine-Grained Execution Feedback and Cost-Efficient Rewards*. IEEE ICDE 2026 (arXiv:2605.03465).
4. Huang, H., LeCun, Y., & Balestriero, R. (2026). *LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures*. ICLR 2026 (arXiv:2509.14252).
5. Sheng, L., & Xu, S.-S. (2025b). *CSC-SQL: Corrective Self-Consistency in Text-to-SQL via Reinforcement Learning*. Preprint (arXiv:2505.13271).
6. Zhang, Y., et al. (2026). *Reward-SQL: Stepwise Process-Supervised Reinforcement Learning via Chain-of-CTEs*. ACM SIGMOD 2026 (arXiv:2505.04671).
7. COLM Authors (2026). *Goal-Conditioned Hint Curriculum for Preventing Advantage Collapse in GRPO*. COLM 2026 (OpenReview ID: JWUF2pDVY8).
8. Yao, Z., et al. (2025). *Arctic-Text2SQL-R1: Simple Rewards for Strong Reasoning in Text-to-SQL*. Snowflake AI Research (arXiv:2505.20315).
9. TianyuCodings (2026). *NanoJev: A 0.6B Open-Source Parallel Decision Model with Zero Output-Token Decoding*. GitHub Repository.
10. ConvAI Benchmark Authors (2026). *Laya, CLEF, Kev, and Open System-One Decision Models: ModernBERT Cross-Encoder with RLCD*. Technical Benchmark Reports.